

Unit-5 oops

Easy Questions (2 marks each)

Q1. What is an Applet?

An Applet is a small Java program that runs within a web browser or an applet viewer. It is embedded in HTML and executed by a Java-enabled browser.

Q2. Name two key features of AWT (Abstract Window Toolkit).

1. AWT components are platform-dependent.
 2. Provides a set of pre-built UI components like buttons, labels, and text fields.
-

Q3. List three basic components of Swing.

1. JButton
 2. JLabel
 3. JTextField
-

Q4. What does MVC stand for in the context of GUI design?

MVC stands for **Model-View-Controller**. It is a design pattern that separates data (Model), user interface (View), and control logic (Controller).

Q5. Briefly describe the purpose of a JTextField in Swing.

A **JTextField** is a Swing component that allows the user to enter or edit a single line of text.

Q6. What is the function of a Checkbox in a graphical user interface?

A **Checkbox** allows users to make a binary choice (selected or not selected) as part of a form or dialog.

Q7. How is a frame created in AWT? Name the primary class used.

A frame is created using the **Frame** class in AWT. Example:

```
Frame frame = new Frame("Title");
```

Q8. What is a "pluggable look and feel" in Swing?

It allows Swing applications to change their appearance (look and feel) at runtime without affecting functionality.

Q9. What is the main difference between AWT and Swing?

AWT components are **platform-dependent** (use native OS resources), while Swing components are **platform-independent** and written entirely in Java.

Q10. Name any two layout managers available in AWT.

1. BorderLayout
 2. FlowLayout
-

Q11. What package do Applet classes belong to in Java?

Applet classes belong to the **java.applet** package.

Q12. Name one major disadvantage of using AWT over Swing.

AWT is platform-dependent, leading to inconsistencies in the appearance and behavior across different operating systems.

Q13. What is the purpose of the `init()` method in an Applet?

The `init()` method is used to initialize the Applet and is called once when the Applet is loaded.

Q14. How does a `JButton` differ from a `Button` in AWT?

JButton is a Swing component and platform-independent, while **Button** is an AWT component and platform-dependent.

Q15. What is the `paint()` method used for in Applets?

The `paint()` method is used to render or redraw the graphics on the applet's display area.

Q16. State one advantage of using Swing over AWT.

Swing components are lightweight and provide more features and flexibility compared to AWT.

Q17. Which class in Swing is used to create a window with a title bar?

The **JFrame** class in Swing is used to create a window with a title bar.

Q18. What is the primary purpose of the add() method in AWT and Swing?

The add() method is used to add components to a container like a Frame or Panel.

Q19. Define the term "Container" in the context of AWT and Swing.

A **Container** is a component that can hold other components (child components). Examples: Panel, Frame, JPanel.

Q20. What package do Applet classes belong to in Java?

Applet classes belong to the **java.applet** package. *(Repeated question from Q11.)*

Intermediate Questions (5 marks each)

Q21. Explain the MVC architecture. How is it implemented in Java GUI applications?

The **Model-View-Controller (MVC)** architecture is a design pattern used to separate concerns in applications.

- **Model:** Manages the data and business logic of the application.
- **View:** Displays the data (UI) to the user and reflects changes from the model.
- **Controller:** Handles user input and updates the model or view accordingly.

In Java GUI:

1. **Model:** Data classes or backend logic.
2. **View:** Swing components like JFrame, JPanel, etc.
3. **Controller:** Event listeners (e.g., ActionListener) to respond to user actions.

Example: A JButton triggers an ActionEvent, updating a JLabel via the controller.

Q22. Describe the life cycle of an Applet. Name and explain each phase.

The Applet life cycle includes the following methods:

1. **init():** Called once when the Applet is initialized; used for setup.
2. **start():** Called every time the Applet becomes active or visible.
3. **paint(Graphics g):** Called to redraw the Applet; used for rendering visuals.

4. **stop()**: Called when the Applet becomes inactive or hidden.
 5. **destroy()**: Called when the Applet is permanently removed; used for cleanup.
-

Q23. Write a simple code snippet to create a frame in AWT with a title and dimensions.

```
import java.awt.*;

public class SimpleFrame {

    public static void main(String[] args) {

        Frame frame = new Frame("My AWT Frame");

        frame.setSize(400, 300); // Set dimensions

        frame.setVisible(true); // Make it visible

    }

}
```

Q24. How does JRadioButton differ from JCheckBox in terms of functionality?

- **JRadioButton:**
 - Used for mutually exclusive options (only one can be selected at a time).
 - Often grouped using ButtonGroup.
 - Example: Selecting a gender (Male/Female).
 - **JCheckBox:**
 - Allows multiple selections independently.
 - Example: Selecting hobbies (Reading, Cooking).
-

Q25. What is the significance of the EventListener interface in AWT and Swing? Provide an example.

The **EventListener** interface is part of Java's event-handling mechanism. It is implemented by classes to respond to specific events like button clicks, key presses, or mouse actions.

Example: Handling a button click with ActionListener.

```
import javax.swing.*;

import java.awt.event.*;
```

```

public class EventExample {

    public static void main(String[] args) {

        JFrame frame = new JFrame("Event Listener Example");

        JButton button = new JButton("Click Me");

        button.addActionListener(new ActionListener() {

            public void actionPerformed(ActionEvent e) {

                System.out.println("Button clicked!");

            }

        });

        frame.add(button);

        frame.setSize(200, 100);

        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        frame.setVisible(true);

    }

}

```

Q26. Explain how Swing components are lightweight and why this is advantageous.

Swing components are considered **lightweight** because they are written entirely in Java and do not rely on native OS resources.

Advantages:

1. **Platform independence:** Uniform appearance across platforms.
2. **Customizable:** Supports "pluggable look and feel."
3. **Better performance:** Reduced dependency on native libraries.

Q27. Create a Swing interface with a JButton and JLabel. Set an action to change the label text when the button is clicked.

```

import javax.swing.*.*;

import java.awt.event.*;

```

```

public class SwingExample {

    public static void main(String[] args) {

        JFrame frame = new JFrame("Swing Example");

        JLabel label = new JLabel("Old Text");

        JButton button = new JButton("Click Me");

        button.addActionListener(new ActionListener() {

            public void actionPerformed(ActionEvent e) {

                label.setText("New Text");

            }

        });

        frame.setLayout(new java.awt.FlowLayout());

        frame.add(label);

        frame.add(button);

        frame.setSize(300, 150);

        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        frame.setVisible(true);

    }

}

```

Q28. Explain how the "pluggable look and feel" works in Swing. How can it be changed at runtime?
 Swing supports changing the UI's look and feel dynamically. The **UIManager** class allows developers to switch styles.

Example:

```

import javax.swing.*.*;

public class LookAndFeelExample {

    public static void main(String[] args) {

```

```

try {
    // Set Nimbus Look and Feel
    UIManager.setLookAndFeel("javax.swing.plaf.nimbus.NimbusLookAndFeel");
} catch (Exception e) {
    e.printStackTrace();
}

JFrame frame = new JFrame("Look and Feel Example");
JButton button = new JButton("Button");
frame.add(button);
frame.setSize(200, 100);
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setVisible(true);
}
}

```

Q29. Describe the primary function of the Container and Component classes in AWT and Swing.

- **Component:** Represents a GUI element (e.g., Button, Label, TextField). It is the base class for all GUI components.
- **Container:** A special component that holds other components, arranging them using layout managers. Examples: Frame, Panel, JPanel.

Q30. Write a code snippet demonstrating how to add a JToggleButton to a Swing application.

```

import javax.swing.*.*;

public class ToggleButtonExample {
    public static void main(String[] args) {
        JFrame frame = new JFrame("JToggleButton Example");
        JToggleButton toggleButton = new JToggleButton("OFF");
    }
}

```

```

toggleButton.addItemListener(e -> {
    if (toggleButton.isSelected()) {
        toggleButton.setText("ON");
    } else {
        toggleButton.setText("OFF");
    }
});
frame.add(toggleButton);
frame.setSize(200, 100);
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setVisible(true);
}
}

```

Difficult Questions (10 marks each)

Q31. Discuss the advantages and disadvantages of using the MVC pattern in GUI applications. Provide examples relevant to AWT/Swing.

Advantages:

1. **Separation of Concerns:** Easier to maintain and scale applications since the model, view, and controller are independent.
2. **Reusability:** Models can be reused across different views or applications.
3. **Testability:** Business logic (model) can be tested independently of the user interface (view).
4. **Flexibility:** Different developers can work simultaneously on the model, view, and controller.

Disadvantages:

1. **Complexity:** Adds an additional layer of abstraction, making small applications unnecessarily complicated.
2. **Overhead:** Requires extra code to establish connections between components.
3. **Learning Curve:** Developers need to understand the pattern thoroughly to implement it effectively.

Example in Java:

In Swing:

- **Model:** A class storing data (e.g., a List<String>).
- **View:** A GUI component (e.g., JTable) displaying the data.
- **Controller:** An ActionListener that updates the model when a button is clicked.

java

Copy code

```
// Example of MVC in a simple calculator
// Model: Logic to perform calculations
// View: Swing GUI (buttons, text fields)
// Controller: Handles button clicks to update the result
```

Q32. Write a program in Java to implement GUI in AWT.

java

Copy code

```
import java.awt.*;
import java.awt.event.*;

public class AWTEExample {
    public static void main(String[] args) {
        Frame frame = new Frame("AWT Example");
        Label label = new Label("Enter your name:");
        TextField textField = new TextField(20);
        Button button = new Button("Submit");
        Label result = new Label();

        button.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                result.setText("Hello, " + textField.getText() + "!");
            }
        });
    }
}
```

```

frame.setLayout(new FlowLayout());

frame.add(label);

frame.add(textField);

frame.add(button);

frame.add(result);


frame.setSize(300, 200);

frame.setVisible(true);


frame.addWindowListener(new WindowAdapter() {
    public void windowClosing(WindowEvent we) {
        System.exit(0);
    }
});
}
}

```

Q33. Demonstrate the life cycle of an Applet with an example code that overrides init(), start(), stop(), and destroy() methods.

java

Copy code

```

import java.applet.Applet;

import java.awt.Graphics;


public class AppletLifecycle extends Applet {
    public void init() {
        System.out.println("Applet Initialized");
    }
}

```

```
public void start() {  
    System.out.println("Applet Started");  
}
```

```
public void paint(Graphics g) {  
    System.out.println("Painting Applet");  
    g.drawString("Hello, Applet!", 50, 50);  
}
```

```
public void stop() {  
    System.out.println("Applet Stopped");  
}
```

```
public void destroy() {  
    System.out.println("Applet Destroyed");  
}  
}
```

To run this Applet, embed it in an HTML file:

html

Copy code

```
<applet code="AppletLifecycle.class" width="300" height="150"></applet>
```

Q34. Explain the key differences between AWT and Swing components. Give examples of at least five components available in both AWT and Swing, describing how they differ.

Feature	AWT	Swing
Dependency	Platform-dependent	Platform-independent
Performance	Uses native OS resources	Lightweight and faster

Feature	AWT	Swing
Customization	Limited	Highly customizable
Look and Feel	OS-defined	Pluggable look and feel
Components	Basic (e.g., Button)	Enhanced (e.g., JButton)

Examples:

1. **Button** (AWT) vs. **JButton** (Swing): Swing offers more features like icons and tooltips.
2. **Label** (AWT) vs. **JLabel** (Swing): Swing supports HTML rendering.
3. **TextField** (AWT) vs. **JTextField** (Swing): Swing allows advanced formatting.
4. **Frame** (AWT) vs. **JFrame** (Swing): Swing supports better UI customization.
5. **Checkbox** (AWT) vs. **JCheckBox** (Swing): Swing provides better integration with ButtonGroups.

Q35. Create a Java Applet that displays "Hello, World!" in the center of the Applet window. Describe each method you used in the code.

java

Copy code

```
import java.applet.Applet;
import java.awt.Graphics;
```

```
public class HelloWorldApplet extends Applet {
    public void paint(Graphics g) {
        // Draw "Hello, World!" at the center
        g.drawString("Hello, World!", getWidth() / 2 - 50, getHeight() / 2);
    }
}
```

Explanation:

1. **paint(Graphics g)**: Used to draw graphics or text on the applet.
 2. **getWidth()/getHeight()**: Fetch the dimensions of the applet window to center the text.
-

Q36. Explain the difference between heavyweight and lightweight components in AWT and Swing. Provide examples of each and describe scenarios where one type is preferable over the other.

Heavyweight Components:

- **Definition:** Depend on native OS resources (e.g., AWT components).
- **Example:** Button, Label, TextField (AWT).
- **Advantages:**
 1. Faster on older systems with limited Java support.
 2. Direct integration with native system UI.
- **Disadvantages:**
 1. Platform-dependent appearance.
 2. Limited customizability.

Lightweight Components:

- **Definition:** Entirely implemented in Java without reliance on OS resources (e.g., Swing components).
- **Example:** JButton, JLabel, JTextField (Swing).
- **Advantages:**
 1. Platform-independent.
 2. Highly customizable with pluggable look and feel.
- **Disadvantages:**
 1. Slightly slower on older hardware.

Scenarios:

- **Heavyweight:** Legacy applications requiring native look and feel.
- **Lightweight:** Modern, cross-platform GUIs with rich customizations.